

第五部分：LLM 与大模型应用 — 农业知识问答系统

概述

本部分将大语言模型（LLM）应用到真实的农业场景：**构建一个农业知识问答系统**。你将学习：

- 如何通过 API 调用大语言模型，掌握 Prompt Engineering 技巧
- 如何从网络采集农业知识数据，构建自己的知识库
- 如何搭建 RAG（检索增强生成）系统，让 LLM 回答更专业、更准确
- 如何在 GPU 服务器上用 vLLM 部署开源模型
- 如何整合所有技能，构建端到端的农业 AI 助手

应用场景：农民或农业技术人员输入问题（如“番茄叶子发黄是什么原因？”），系统自动检索相关知识库文档，由 LLM 生成专业、有依据的回答。

环境准备

第一步：确认你的环境

配置	用途	说明
Docker 环境 （沿用前4部分的 dl-env ）	Task A/B/D	Python + Jupyter 环境
agicto API Key	Task A/B/D	调用云端 LLM，需提前注册获取
云端 3090 GPU 服务器	Task C	SSH 远程连接，部署 vLLM

第二步：获取 agicto API Key

1. 访问 agicto 平台注册账号
2. 在控制台中获取 API Key
3. 将 API Key 保存到环境变量或配置文件中（不要硬编码在代码里）

```
# 推荐方式：通过环境变量读取
import os
api_key = os.environ.get("AGICTO_API_KEY", "your-api-key-here")
```

base_url: <https://api.agicto.cn/v1>（兼容 OpenAI SDK 接口）

第三步：启动 Docker + Jupyter

```
cd 5-LLM与大模型应用

# CPU 模式（Task A/B 不需要 GPU）
docker run -it --rm -p 8888:8888 -p 8501:8501 \
-v D:\my-dl-env\5-LLM与大模型应用:/workspace \
```

```
-w /workspace dl-env:latest jupyter notebook \  
--ip=0.0.0.0 --port=8888 --no-browser --allow-root
```

启动后，浏览器打开终端显示的链接（含 token）。

第四步：安装额外依赖

在 Jupyter 的 Terminal 或 Notebook 中运行：

```
pip install openai chromadb sentence-transformers beautifulsoup4 requests  
streamlit flagembedding
```

说明： `flagembedding` 库用于加载 BGE-M3 向量模型（支持 CPU 运行）。

第五步：验证环境

```
import torch  
from openai import OpenAI  
  
client = OpenAI(  
    api_key="your-api-key",  
    base_url="https://api.agicto.cn/v1"  
)  
print("环境配置完成！")
```

LLM 基础知识

什么是大语言模型？

大语言模型（Large Language Model, LLM）是在海量文本上训练的神经网络，能够理解和生成人类语言。你可能已经用过 ChatGPT、文心一言等产品，它们的背后都是 LLM。

核心架构：Transformer

输入文本 → Tokenizer（分词） → Embedding（向量化） → Transformer 层（自注意力 + 前馈）
→ 输出概率分布 → 生成下一个词

- **Token：**模型处理文本的基本单位。中文一个词通常是一个 token。
- **自注意力机制（Self-Attention）：**让模型理解词与词之间的关系，不管它们距离多远。
- **上下文窗口：**模型一次能处理的 token 数量。越大越好，但也更耗资源。

训练过程（简要）



任务 A：LLM API 初体验与 Prompt Engineering（30 分钟）

学习目标

- 学会调用 agicto API 进行对话
- 掌握 Prompt Engineering 基本技巧
- 在农业场景下实践 prompt 设计

背景

Prompt Engineering（提示词工程）是与 LLM 交互的核心技能。**好的 prompt 能让模型输出更准确、更有用的内容**，差的 prompt 则可能得到模糊甚至错误的回答。

差的 prompt: "番茄病害怎么办？"

→ 回答太笼统

好的 prompt: "你是农业专家。番茄叶片出现黄色斑点，边缘有黄晕，主要发生在中下部叶片。请分析可能的病害类型、致病原因和防治措施。请用结构化的方式回答。"

→ 回答专业、有针对性

Prompt 设计模式

模式	说明	示例
角色设定	给模型指定身份	"你是一位有20年经验的农业植保专家"
Few-shot	给几个输入输出示例	给出 2-3 个"症状 → 诊断"的例子
Chain of Thought	引导模型逐步推理	"请先分析症状，再列出可能原因，最后给出建议"
结构化输出	指定回答格式	"请按以下格式回答：1. 病害名称 2. 病因 3. 症状 4. 防治措施"

任务步骤

在 Jupyter 中新建 `task_a_llm_api.ipynb`：

步骤 1：配置 API 客户端

```
import os
from openai import OpenAI

# 配置 agicto API
client = OpenAI(
    api_key=os.environ.get("AGICTO_API_KEY", "替换为你的API Key"),
    base_url="https://api.agicto.cn/v1"
)

MODEL = "gpt-4o-mini" # 或使用 "qwen-plus" 等其他模型
```

步骤 2：基础对话

```
def chat_with_llm(messages, model=MODEL, temperature=0.7, max_tokens=1024):
    """发送消息给 LLM 并获取回复"""
    response = client.chat.completions.create(
        model=model,
        messages=messages,
        temperature=temperature,
        max_tokens=max_tokens
    )
    return response.choices[0].message.content

# 基础测试
messages = [{"role": "user", "content": "你好，请简单介绍下番茄的种植要点。"}]
reply = chat_with_llm(messages)
print(reply)
```

步骤 3：Prompt Engineering 实验

比较三种不同的 prompt 策略：

```
import time

def benchmark_prompt(system_prompt, user_prompt, label=""):
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ]
    start = time.time()
    reply = chat_with_llm(messages, temperature=0.3)
    elapsed = time.time() - start
    print(f"\n{'='*60}")
    print(f"【{label}】")
    print(f"耗时: {elapsed:.1f}s")
    print(f"回答:\n{reply}")
    return reply
```

```
# — 实验 1: 基础 prompt (无角色设定) —
benchmark_prompt(
    system_prompt="",
    user_prompt="番茄叶子发黄是什么原因?",
    label="实验1: 基础Prompt"
)

# — 实验 2: 角色设定 + 结构化要求 —
benchmark_prompt(
    system_prompt="你是一位有20年经验的农业植保专家。请用专业但易懂的语言回答农民的病害问题。",
    user_prompt="番茄叶子发黄是什么原因? 请从以下方面分析: 1. 可能的病害类型 2. 非病害原因 (营养、水分等) 3. 如何区分 4. 防治建议",
    label="实验2: 角色设定+结构化"
)

# — 实验 3: Few-shot + Chain of Thought —
benchmark_prompt(
    system_prompt="你是一位农业植保专家。请按以下步骤分析: 先观察症状特征, 再对比已知病害, 最后给出诊断和建议。",
    user_prompt="""请参考以下诊断示例, 对新的症状进行分析:

示例1:
- 症状: 番茄叶片出现同心轮纹状褐色病斑
- 诊断: 早疫病 (Alternaria solani)。轮纹状病斑是典型特征。
- 建议: 移除病叶, 喷施代森锰锌或苯醚甲环唑。

示例2:
- 症状: 番茄果实出现水渍状褐色凹陷斑, 高湿环境下有白色霉层
- 诊断: 晚疫病 (Phytophthora infestans)。水渍状斑和白色霉层是关键特征。
- 建议: 控制湿度, 喷施烯酰吗啉或霜脲氰。

新症状: 番茄叶片边缘出现 V 形黄褐色坏死斑, 病健交界明显, 多从下部叶片开始。
请逐步分析。""",
    label="实验3: Few-shot+逐步推理"
)
```

步骤 4: 多轮对话

```
# 模拟一次完整的农业咨询对话
messages = [
    {"role": "system", "content": "你是农业植保专家, 专门帮助农民诊断和防治作物病害。回答要专业、实用、有依据。"}
]

def multi_turn_chat(user_input):
    messages.append({"role": "user", "content": user_input})
    reply = chat_with_llm(messages, temperature=0.5)
    messages.append({"role": "assistant", "content": reply})
    print(f"你: {user_input}")
```

```
print(f"专家: {reply}\n")
return reply

# 多轮对话测试
multi_turn_chat("我家的番茄最近叶子开始出现褐色斑点，集中在下部叶片，是怎么回事？")
multi_turn_chat("用什么药比较好？我现在手头有代森锰锌。")
multi_turn_chat("打药的频率应该是多少？")
```

步骤 5：可视化对比

```
import matplotlib.pyplot as plt

# 记录实验数据
experiments = ["基础Prompt", "角色+结构化", "Few-shot+推理"]
quality_scores = [3, 7, 9] # 主观评分 1-10，基于回答的专业性和实用性

fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.barh(experiments, quality_scores, color=["#4e79a7", "#f28e2b", "#e15759"], edgecolor="black")
for bar, score in zip(bars, quality_scores):
    ax.text(score + 0.2, bar.get_y() + bar.get_height()/2,
            f"{score}/10", va="center", fontsize=12, fontweight="bold")
ax.set_xlabel("回答质量评分 (主观)")
ax.set_title("Prompt Engineering 效果对比")
ax.set_xlim(0, 11)
ax.grid(True, alpha=0.3, axis="x")
plt.tight_layout()
plt.savefig("task_a_prompt_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
```

思考题

1. 为什么角色设定能显著提升 LLM 的回答质量？
2. Few-shot learning 中，示例的数量和质量哪个更重要？
3. 在农业场景下，结构化输出为什么特别重要？

任务 B：RAG 农业知识库问答系统（40 分钟）

学习目标

- 理解 RAG（检索增强生成）的原理与价值
- 学会从网络采集农业知识数据
- 掌握知识库构建方法
- 构建端到端的农业知识问答 Pipeline

背景

为什么需要 RAG？

LLM 虽然强大，但有三个关键问题：

1. **幻觉 (Hallucination)**：模型会编造看似合理但实际错误的信息
2. **知识时效性**：模型训练截止于某个时间点，无法获取最新知识
3. **领域专业性**：通用模型对特定领域（如农业）的知识不够深入

RAG (Retrieval-Augmented Generation) 的解决方案：

用户提问 → 检索知识库找到相关文档 → 将文档作为上下文给 LLM → LLM 基于事实生成回答

这样 LLM 不再“凭记忆瞎猜”，而是“看着参考资料回答”。

第一部分：数据采集 — 从网络获取农业知识

农业知识可以从多个公开渠道获取。下面教你用简单的 Python 爬虫采集网页内容。

数据采集注意事项

- **法律边界**：只采集公开数据，尊重 `robots.txt`，不要绕过登录验证
- **频率限制**：不要高频请求服务器，建议每次请求间隔 2-3 秒
- **版权意识**：采集的内容用于学习和研究，不要商用

示例：爬取农业知识文章

创建 `scrape_example.py`：

重要说明：下面的脚本只是一个**示例和起点**，它使用通用的 HTML 解析策略（查找 `<article>`、`<div class="content">` 或 `<p>` 标签）。**不同的网站结构千差万别**，这个脚本不一定能直接适配你要爬取的网站。你需要：

- 用浏览器的开发者工具（F12）观察目标网页的 HTML 结构
- 根据实际结构修改 `extract_article()` 中的查找逻辑
- 有些网站使用了 JavaScript 动态加载内容，`requests` 无法获取，需要借助 Selenium 或 Playwright
- 有些网站有反爬机制（验证码、IP 限制、Cookie 验证等），需要额外处理

探索建议：网上有大量 Python 爬虫教程，搜索 "Python 网页爬虫 BeautifulSoup 教程" 或 "Python 爬虫入门"，结合你要抓取的网站自行学习和调整。

```
import requests
from bs4 import BeautifulSoup
import time
import os

def fetch_page(url, encoding="utf-8"):
    """获取网页内容"""
    headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
```

```
AppleWebKit/537.36"
}
try:
    response = requests.get(url, headers=headers, timeout=10)
    # 有些中文网页用 GBK 编码
    if encoding == "auto":
        response.encoding = response.apparent_encoding
    else:
        response.encoding = encoding
    return response.text
except Exception as e:
    print(f"请求失败: {e}")
    return None

def extract_article(html):
    """从网页中提取正文（简化版）"""
    soup = BeautifulSoup(html, "html.parser")

    # 删除脚本、样式、导航等无关元素
    for tag in soup(["script", "style", "nav", "footer", "header", "aside"]):
        tag.decompose()

    # 尝试获取正文区域（不同网站结构不同，这里用通用方法）
    article = soup.find("article") or soup.find("div", class_="content") or
    soup.find("main")

    if article:
        text = article.get_text(separator="\n", strip=True)
    else:
        # 退而求其次，获取所有段落
        paragraphs = soup.find_all("p")
        text = "\n".join([p.get_text(strip=True) for p in paragraphs if
len(p.get_text(strip=True)) > 50])

    # 清理多余空行
    lines = [line.strip() for line in text.split("\n") if line.strip()]
    return "\n\n".join(lines)

# — 示例用法 —
if __name__ == "__main__":
    # 示例 URL（需要替换为实际可抓取的农业知识网页）
    urls = [
        # "https://example-agri-site.com/tomato-disease-guide",
        # "https://example-agri-site.com/planting-guide",
    ]

    if not urls:
        print("请在代码中添加实际的农业知识网页 URL")
        print("推荐来源：中国农业信息网、各省市农业厅公开资料等")
        print("\n示例流程：")
        print("1. 找到一篇关于番茄病害的科普文章")
        print("2. 使用 fetch_page() 获取网页 HTML")
        print("3. 使用 extract_article() 提取正文")
        print("4. 保存到 knowledge_base/ 目录下")
```



```
else:
    os.makedirs("knowledge_base", exist_ok=True)
    for i, url in enumerate(urls):
        print(f"正在抓取第 {i+1} 篇文章...")
        html = fetch_page(url, encoding="auto")
        if html:
            text = extract_article(html)
            filename = f"knowledge_base/article_{i+1}.md"
            with open(filename, "w", encoding="utf-8") as f:
                f.write(f"# 第{i+1}篇文章\n\n")
                f.write(f"来源: {url}\n\n")
                f.write(text)
            print(f"已保存到 {filename}")
        time.sleep(2) # 礼貌延迟
```

练习： 找一个公开的农业知识网页（如中国农业信息网的科普文章），运行上面的脚本抓取内容。

第二部分：知识库构建

知识库目录结构

```
knowledge_base/
├── diseases/           # 病害知识
│   ├── tomato_early_blight.md      # 番茄早疫病
│   ├── tomato_late_blight.md       # 番茄晚疫病
│   └── tomato_bacterial_spot.md     # 番茄细菌性斑点病
├── planting/           # 种植技术
│   └── tomato_planting_guide.md     # 番茄种植指南
├── pesticide/          # 农药使用
│   └── common_pesticides.md         # 常用农药手册
```

文档编写规范

- **标题清晰：** 每个文档一个主题
- **内容结构化：** 使用 Markdown 标题、列表、表格
- **标注来源：** 每篇文档注明出处
- **篇幅适中：** 每篇 500-2000 字，按主题拆分

预填充示例文档

在 `knowledge_base/diseases/tomato_early_blight.md` 中：

```
# 番茄早疫病 (Alternaria Leaf Spot)

## 基本信息
- **病原**：链格孢菌 (Alternaria solani)
- **发病条件**：温度 20-25°C，相对湿度 80% 以上，多雨季节易发
```

- ****危害部位****: 主要危害叶片，也可危害茎秆和果实

症状识别

1. ****叶片****: 初为水渍状暗褐色小点，后扩大为圆形或近圆形病斑
2. ****特征性病斑****: 褐色同心轮纹（像靶心），边缘有黄色晕圈
3. ****严重情况****: 病斑连片，叶片枯黄脱落，从下部叶片开始

发病规律

- 病菌以菌丝体或分生孢子在病残体、种子中越冬
- 第二年借风雨传播，从气孔或伤口侵入
- 连作地、排水不良、植株长势弱的地块发病重

防治措施

农业防治

- 轮作倒茬（与非茄科作物轮作 2-3 年）
- 清除病残体，减少初侵染源
- 合理密植，改善通风透光条件
- 科学施肥，增强植株抗病力

化学防治

药剂	浓度	用法	安全间隔期
-----	-----	-----	-----
代森锰锌	600-800 倍液	喷雾	15 天
苯醚甲环唑	1500-2000 倍液	喷雾	7 天
百菌清	500-600 倍液	喷雾	10 天

> ****注意****: 化学防治应及早进行，发病初期每 7-10 天喷一次，连续 2-3 次。

来源

《蔬菜病虫害防治手册》（第三版），中国农业出版社

第三部分：知识库增强 — 让 RAG 更聪明

在基本 RAG 之上，有很多可以改进的方向。**不要满足于"能跑就行"**，思考如何让系统变得更智能。

3.1 更好的分块策略

上面的 `chunk_text` 函数按段落分块，但你可以尝试更多策略：

分块策略	适用场景	说明
固定大小	通用场景	每块固定 500 字，重叠 50 字
按标题分块	结构化文档	每个二级标题（##）下的内容作为一块
按语义分块	长文档	用句子边界分块，保持语义完整
重叠增强	关键信息在边界	块间重叠 10-20%，减少信息丢失

练习：试试按 Markdown 标题（## ）分块：

```
def chunk_by_headings(text):  
    """按 Markdown 二级标题分块"""  
    sections = text.split("## ")  
    chunks = []  
    for section in sections:  
        section = section.strip()  
        if section:  
            chunks.append("## " + section) # 补回标题标记  
    return chunks
```

思考：哪种分块策略最适合农业知识文档？为什么？

3.2 元数据增强检索

给每个 chunk 添加额外的元数据，可以大幅提升检索精度：

```
# 在分块时提取关键词作为元数据  
chunk_meta = {  
    "source": doc["source"],  
    "title": doc["title"],  
    "category": "disease", # 手动或自动标注类别  
    "keywords": ["早疫病", "番茄", "病斑"], # 关键词  
}
```

检索时可以利用这些元数据进行过滤，比如只在病害类文档中检索。

3.3 多粒度检索

不只是检索 chunk，可以同时检索不同粒度的内容：

- **段落级**：精确匹配具体细节
- **文档级**：获取整体上下文
- **混合检索**：同时检索不同粒度的索引，合并结果

3.4 查询改写

用户的提问可能不够精确，可以在检索前先改写查询：

```
def rewrite_query(question, client, model):  
    """用 LLM 改写用户查询，提升检索效果"""  
    messages = [  
        {"role": "system", "content": "你是一个查询优化助手。将用户的问题改写为 3 个不同的版本，适合用于农业知识库检索。"},  
        {"role": "user", "content": f"请改写以下查询，使其更适合农业知识库检索：{question}"},  
    ]  
    response = client.chat.completions.create(  
        messages=messages,  
        model=model,  
    )
```

```
        model=model, messages=messages, temperature=0.3, max_tokens=256
    )
    return response.choices[0].message.content
```

然后用改写后的多个查询分别检索，合并结果。

3.5 评估你的 RAG 系统

好的 RAG 系统需要评估，不只是“看起来可以”：

- **检索准确率**：检索到的 chunk 是否真的相关？（人工标注 20 个测试问题）
- **回答质量**：基于检索的回答是否比直接回答更好？
- **端到端评估**：用户满意度如何？

第四部分：RAG 系统实现

RAG 架构概览

```
知识文档 → 文本分块 (chunk) → 向量化 (embedding) → 存入向量数据库
                                                    ↓
用户提问 → 向量化 → 在数据库中检索最相关的 chunk → 拼接为 prompt → LLM 生成回答
```

步骤 1：安装和配置

```
import os
from openai import OpenAI
import chromadb
from chromadb.utils import embedding_functions
from sentence_transformers import SentenceTransformer
import numpy as np

# agicto API 客户端
client = OpenAI(
    api_key=os.environ.get("AGICTO_API_KEY", "替换为你的API Key"),
    base_url="https://api.agicto.cn/v1"
)
MODEL = "qwen-plus"

# 加载 BGE-M3 向量模型（支持 CPU 运行）
# BGE-M3 是智源研究院的开源多语言模型，中英文都表现优秀
print("加载 BGE-M3 向量模型（首次下载约 2GB，请耐心等待）...")
embedder = SentenceTransformer("BAAI/bge-m3", trust_remote_code=True)
print("向量模型加载完成！")
```

关于 BGE-M3：这是一个支持 CPU 运行的中文向量模型，约 2GB。如果你的网络较慢，首次加载可能需要几分钟。模型会缓存到本地，后续加载很快。

步骤 2：加载知识库文档

```
import glob

def load_documents(directory):
    """加载指定目录下的所有 .md / .txt 文件"""
    documents = []
    file_paths = glob.glob(os.path.join(directory, "**/*.md"), recursive=True)
    file_paths += glob.glob(os.path.join(directory, "**/*.txt"), recursive=True)

    for path in file_paths:
        with open(path, "r", encoding="utf-8") as f:
            content = f.read()
            rel_path = os.path.relpath(path, directory)
            documents.append({
                "source": rel_path,
                "content": content,
                "title": os.path.basename(path)
            })

    print(f"加载了 {len(documents)} 篇文档")
    for doc in documents:
        print(f"  - {doc['source']} ({len(doc['content'])} 字符)")

    return documents

docs = load_documents("knowledge_base")
```

步骤 3：文档分块 (Chunking)

```
def chunk_text(text, chunk_size=500, overlap=50):
    """将长文本分成重叠的块"""
    chunks = []
    # 先按标题/段落分割
    paragraphs = text.split("\n\n")

    current_chunk = ""
    for para in paragraphs:
        para = para.strip()
        if not para:
            continue

        if len(current_chunk) + len(para) < chunk_size:
            current_chunk += "\n\n" + para if current_chunk else para
        else:
            if current_chunk:
                chunks.append(current_chunk)
            # 保留一部分重叠内容，保持上下文连贯
            words = current_chunk.split()
            overlap_text = " ".join(words[-overlap:]) if len(words) > overlap else ""
```

```
current_chunk
    current_chunk = overlap_text + "\n\n" + para

    if current_chunk:
        chunks.append(current_chunk)

    return chunks

# 对所有文档分块
all_chunks = []
for doc in docs:
    chunks = chunk_text(doc["content"], chunk_size=500, overlap=50)
    for i, chunk in enumerate(chunks):
        all_chunks.append({
            "text": chunk,
            "source": doc["source"],
            "title": doc["title"],
            "chunk_id": f"{doc['source']}_{i}"
        })

print(f"共分成 {len(all_chunks)} 个文本块")
```

步骤 4：构建向量数据库

```
# 初始化 ChromaDB
chroma_client = chromadb.PersistentClient(path="./chroma_db")

# 使用 BGE-M3 embedding 模型
embedding_func = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="BAAI/bge-m3"
)

# 创建或获取集合
collection_name = "agri_knowledge"
try:
    collection = chroma_client.get_collection(name=collection_name,
        embedding_function=embedding_func)
    print(f"使用已有集合: {collection_name}")
except Exception:
    collection = chroma_client.create_collection(
        name=collection_name,
        embedding_function=embedding_func,
        metadata={"description": "农业知识库"}
    )
    print(f"创建新集合: {collection_name}")

# 清空并添加数据
collection.delete(where={}) # 清空旧数据

texts = [chunk["text"] for chunk in all_chunks]
ids = [chunk["chunk_id"] for chunk in all_chunks]
```

```

metadatas = [
    {"source": chunk["source"], "title": chunk["title"]}
    for chunk in all_chunks
]

# 分批添加（避免一次太多）
batch_size = 100
for i in range(0, len(texts), batch_size):
    batch_end = min(i + batch_size, len(texts))
    collection.add(
        documents=texts[i:batch_end],
        ids=ids[i:batch_end],
        metadatas=metadatas[i:batch_end]
    )

print(f"已向量化并存储 {len(texts)} 个文本块")

```

步骤 5：检索与问答

```

def retrieve_and_answer(question, top_k=3):
    """检索相关知识并生成回答"""
    # — 第一步：检索 —
    results = collection.query(
        query_texts=[question],
        n_results=top_k
    )

    print(f"\n问题: {question}")
    print(f"\n检索到的相关知识:")
    print("-" * 50)

    # 整理检索到的内容
    context_parts = []
    for i, (doc, metadata, distance) in enumerate(zip(
        results["documents"][0],
        results["metadatas"][0],
        results["distances"][0]
    )):
        print(f"\n来源: {metadata['source']} (相关度: {1-distance:.2f})")
        print(f"内容: {doc[:200]}...")
        context_parts.append(f"来源: {metadata['source']}\n内容: {doc}")

    # — 第二步：构建 prompt —
    context = "\n\n---\n\n".join(context_parts)

    system_prompt = """你是农业植保专家。请基于以下参考资料回答用户的问题。
要求：
1. 回答必须基于参考资料，不要编造信息
2. 如果参考资料不足以回答问题，请如实告知
3. 回答要实用、有针对性
4. 在回答末尾注明参考来源"""

```

```

    user_prompt = f"""参考资料:
{context}

用户问题: {question}

请基于以上资料, 给出专业、实用的回答。"""

# — 第三步: 生成回答 —
messages = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": user_prompt}
]
answer = client.chat.completions.create(
    model=MODEL,
    messages=messages,
    temperature=0.3,
    max_tokens=1024
).choices[0].message.content

print(f"\n{'='*50}")
print(f"AI 回答:")
print("-" * 50)
print(answer)

return answer

# — 测试问答 —
questions = [
    "番茄早疫病症状和防治方法是什么?",
    "番茄叶片出现褐色斑点可能是什么病?",
    "代森锰锌的使用方法和注意事项是什么?",
]

for q in questions:
    retrieve_and_answer(q, top_k=3)
    print("\n" + "=" * 60)

```

步骤 6: 对比实验 — 有 RAG vs 无 RAG

```

def compare_rag(question):
    """对比有/无 RAG 的回答"""
    import matplotlib.pyplot as plt

    # 无 RAG: 直接问 LLM
    messages_no_rag = [
        {"role": "system", "content": "你是农业专家。"},
        {"role": "user", "content": question}
    ]
    answer_no_rag = client.chat.completions.create(
        model=MODEL, messages=messages_no_rag, temperature=0.5, max_tokens=512
    )

```



```
    ).choices[0].message.content

# 有 RAG
answer_with_rag = retrieve_and_answer(question)

fig, ax = plt.subplots(figsize=(10, 6))
ax.axis("off")

text = f"问题: {question}\n\n"
text += f"{'-'*60}\n"
text += f"【无 RAG (凭记忆回答)】\n{answer_no_rag[:500]}...\n\n"
text += f"{'-'*60}\n"
text += f"【有 RAG (基于知识库)】\n{answer_with_rag[:500]}..."

ax.text(0.02, 0.98, text, transform=ax.transAxes, fontsize=10,
        verticalalignment="top", family="monospace",
        bbox=dict(boxstyle="round", facecolor="wheat", alpha=0.3))

plt.tight_layout()
plt.savefig("task_b_rag_comparison.png", dpi=150, bbox_inches="tight")
plt.show()

compare_rag("番茄早疫病用什么药治疗? 推荐剂量是多少? ")
```

思考题

1. RAG 系统为什么能减少 LLM 的"幻觉"问题?
2. 文本分块的大小 (chunk_size) 对检索效果有什么影响? 太大或太小会怎样?
3. 如果知识库中没有相关内容, 系统应该如何处理?
4. 中文向量模型和英文模型有什么区别? 为什么要用专门的中文模型?

任务 C: vLLM 本地部署开源模型 (40 分钟)

学习目标

- 理解模型部署的基本概念
- 学会在 GPU 服务器上用 vLLM 部署开源模型
- 对比云端 API 与本地部署的优劣

vLLM 简介

vLLM 是一个高性能的 LLM 推理引擎, 核心优势:

- **PagedAttention**: 类似操作系统分页内存的技术, 显存利用率提升 4 倍
- **高吞吐**: 批量处理请求, 比原生 HuggingFace 推理快 24 倍
- **OpenAI 兼容接口**: 部署后可以直接用 OpenAI SDK 调用, 和 agicto API 用法一致

连接云端 GPU 服务器

准备工作

你需要从培训组织方获取以下信息：

- 服务器 IP 地址
- 登录用户名
- 密码或 SSH 密钥

VSCode Remote-SSH 连接

本任务需要在 GPU 服务器上部署 vLLM，推荐使用 VSCode 的 Remote-SSH 功能连接服务器。以下是基本步骤，网上有大量详细教程（搜索 "VSCode Remote SSH 教程"），建议你自行探索以加深理解：

1. **安装扩展**：在 VSCode 中搜索并安装 **Remote - SSH** 扩展
2. **添加主机**：点击 VSCode 左下角绿色远程图标（或按 **F1** 输入 **Remote-SSH: Connect to Host**）
3. **输入连接信息**：选择 "Add New SSH Host..."，输入 **ssh 用户名@服务器IP**
4. **认证**：选择配置文件路径，然后输入密码（如果配置了 SSH 密钥则免密）
5. **连接成功**：VSCode 窗口左下角会显示 SSH 连接的主机名，此时终端已在服务器上运行

连接成功后，你可以：

- 在远程服务器上编辑文件
- 打开远程终端
- 运行 Python 代码和 vLLM 服务

备选方案：也可以直接用系统终端 SSH 连接：**ssh 用户名@服务器IP**，然后在服务器上操作。

检查服务器环境

连接服务器后，先检查环境：

```
# 检查 GPU
nvidia-smi

# 检查 CUDA 版本
nvcc --version

# 检查 Python 版本
python3 --version

# 检查磁盘空间
df -h

# 检查已安装的 Python 包
pip3 list
```

vLLM 安装与部署

```
# 安装 vLLM
pip3 install vllm
```

```
# 启动 vLLM 服务（以 Qwen3-8B 为例）
python3 -m vllm.entrypoints.openai.api_server \
    --model Qwen/Qwen3-8B \
    --dtype half \
    --max-model-len 4096 \
    --host 0.0.0.0 \
    --port 8000
```

启动后，vLLM 会提供类似 OpenAI API 的接口：<http://localhost:8000/v1>

任务步骤

在 Jupyter 中新建 `task_c_vllm_deploy.ipynb`（或用 VSCode 在远程服务器上直接运行）：

步骤 1：本地调用 vLLM 服务

```
from openai import OpenAI
import time

# 连接 vLLM 服务（假设服务器 IP 为 your-server-ip, vLLM 端口 8000）
client = OpenAI(
    api_key="vllm-local", # vLLM 不需要 API Key, 但 SDK 要求填一个值
    base_url="http://localhost:8000/v1"
    # 如果是远程连接, 替换为:
    # base_url="http://your-server-ip:8000/v1"
)

# 测试连接
response = client.chat.completions.create(
    model="Qwen/Qwen3-8B",
    messages=[{"role": "user", "content": "你好, 请介绍一下番茄种植的基本要点。"}],
    temperature=0.5,
    max_tokens=512
)
print(response.choices[0].message.content)
```

步骤 2：性能对比 — vLLM vs agicto API

```
import matplotlib.pyplot as plt

def benchmark_api(client, model, prompt, label, n_trials=3):
    """测试 API 的响应速度和输出质量"""
    times = []
    outputs = []

    for i in range(n_trials):
        start = time.time()
```

```

        response = client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            temperature=0.5,
            max_tokens=512
        )
        elapsed = time.time() - start
        times.append(elapsed)
        outputs.append(response.choices[0].message.content)
        print(f" {label} Trial {i+1}: {elapsed:.2f}s")

    avg_time = sum(times) / len(times)
    output_len = sum(len(o) for o in outputs) / len(outputs)

    return {
        "label": label,
        "avg_time": avg_time,
        "avg_output_len": output_len,
        "sample": outputs[0][:200]
    }

# 两个客户端
vllm_client = OpenAI(api_key="vllm-local", base_url="http://localhost:8000/v1")
cloud_client = OpenAI(
    api_key="your-api-key",
    base_url="https://api.agictc.cn/v1"
)

# 测试 prompt
test_prompt = "请详细介绍番茄早疫病的发病原因、症状识别和防治方法，包括农业防治和化学防治的具体措施。"

print("正在测试 vLLM 本地部署...")
vllm_result = benchmark_api(vllm_client, "Qwen/Qwen3-8B", test_prompt, "vLLM 本地")

print("\n正在测试 agictc 云端 API...")
cloud_result = benchmark_api(cloud_client, "qwen-plus", test_prompt, "agictc 云端")

# 对比可视化
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# 响应时间
labels = [vllm_result["label"], cloud_result["label"]]
times = [vllm_result["avg_time"], cloud_result["avg_time"]]
colors = ["#4e79a7", "#e15759"]
bars = ax1.bar(labels, times, color=colors, edgecolor="black")
for bar, t in zip(bars, times):
    ax1.text(bar.get_x() + bar.get_width()/2, t + 0.1,
             f"{t:.2f}s", ha="center", fontsize=12, fontweight="bold")
ax1.set_ylabel("平均响应时间 (s)")
ax1.set_title("响应速度对比")
ax1.grid(True, alpha=0.3, axis="y")

```

```
# 输出长度（反映回答详细程度）
output_lens = [vllm_result["avg_output_len"], cloud_result["avg_output_len"]]
bars = ax2.bar(labels, output_lens, color=colors, edgecolor="black")
for bar, l in zip(bars, output_lens):
    ax2.text(bar.get_x() + bar.get_width()/2, l + 10,
             f"{l:.0f} 字符", ha="center", fontsize=12, fontweight="bold")
ax2.set_ylabel("平均输出长度（字符）")
ax2.set_title("回答详细程度对比")
ax2.grid(True, alpha=0.3, axis="y")

plt.tight_layout()
plt.savefig("task_c_performance_comparison.png", dpi=150, bbox_inches="tight")
plt.show()

# 打印样例输出
print(f"\nvLLM 样例回答（前200字）:\n{vllm_result['sample']}...")
print(f"\nagicto 样例回答（前200字）:\n{cloud_result['sample']}...")
```

步骤 3：模型选型讨论

模型	参数量	显存需求（FP16）	中文能力	特点
Qwen3-8B	8B	~16GB	优秀	阿里最新模型，能力强
Qwen2.5-7B-Instruct	7B	~14GB	优秀	成熟稳定，中文最强
Qwen2.5-14B-Instruct	14B	~28GB	优秀	效果更好，需要更多显存
Llama-3.2-3B-Instruct	3B	~6GB	一般	Meta 出品，轻量级

RTX 3090（24GB 显存）推荐方案：

- Qwen3-8B（FP16，约 16GB 显存）— 最新模型，效果最佳
- Qwen2.5-7B-Instruct（FP16，约 14GB 显存）— 成熟稳定
- Qwen2.5-14B-Instruct（量化到 INT8，约 14GB 显存）— 效果更好但需量化

思考题

1. vLLM 相比直接用 HuggingFace 推理有什么优势？
2. 如果显存不够，有哪些方法可以降低显存占用？（提示：量化、更小的模型）
3. 本地部署和云端 API 各有什么优劣？在实际项目中如何选择？

任务 D：综合项目 — 完整的农业 AI 助手（40 分钟）

学习目标

- 综合运用 LLM API、RAG 等技能
- 构建端到端的农业知识问答 Web 应用
- 学会将 AI 服务暴露给真实用户

背景

前三个任务分别学习了：

- Task A：LLM API 调用和 Prompt Engineering
- Task B：RAG 知识库问答
- Task C：vLLM 远程服务器部署

本任务将 Task B 的 RAG 系统整合为 **Web 应用**，在前4部分构建的 Docker 容器中运行，使用 agicto 云端 API 作为 LLM 后端，让农业技术人员可以通过浏览器直接提问。

任务要求

1. 用 **Streamlit** 构建 Web UI（简单易用的 Python Web 框架）
2. 整合 Task B 的 RAG 系统（BGE-M3 向量模型 + ChromaDB）
3. 使用 agicto 云端 API 作为 LLM 后端（支持切换不同模型）
4. 展示检索到的参考文献来源
5. 用 Git 管理代码

参考代码

在 Docker 容器中创建 `task_d_challenge.py`：

```
import streamlit as st
import os
from openai import OpenAI
import chromadb
from chromadb.utils import embedding_functions

# — 页面配置 —
st.set_page_config(page_title="农业 AI 助手", page_icon="🌿", layout="wide")
st.title("🌿 农业 AI 知识问答系统")
st.caption("基于 RAG 技术，整合农业知识库与大语言模型")

# — 侧边栏：配置 —
st.sidebar.header("系统配置")

api_key = st.sidebar.text_input(
    "API Key", type="password", value=os.environ.get("AGICTO_API_KEY", "")
)
model_name = st.sidebar.selectbox("选择模型", ["qwen-plus", "gpt-4o-mini"])
top_k = st.sidebar.slider("检索文档数 (top_k)", 1, 5, 3)

base_url = "https://api.agicto.cn/v1"

# — 初始化客户端 —
@st.cache_resource
def init_clients(_api_key, _base_url):
    llm_client = OpenAI(api_key=_api_key, base_url=_base_url)
    embedder = embedding_functions.SentenceTransformerEmbeddingFunction(
        model_name="BAAI/bge-m3"
    )
```

```

chroma_client = chromadb.PersistentClient(path="./chroma_db")
try:
    collection = chroma_client.get_collection(name="agri_knowledge",
embedding_function=embedder)
except Exception:
    collection = None
    st.sidebar.warning("知识库未初始化, 请先运行 Task B 构建知识库")
return llm_client, collection

llm_client, collection = init_clients(api_key, base_url)

# — 检索函数 —
def retrieve_knowledge(question, top_k=3):
    if collection is None:
        return []
    results = collection.query(
        query_texts=[question],
        n_results=top_k
    )
    return list(zip(results["documents"][0], results["metadatas"][0],
results["distances"][0]))

# — 问答函数 —
def ask_with_rag(question, top_k=3):
    # 检索
    knowledge = retrieve_knowledge(question, top_k)

    if not knowledge:
        # 无知识库, 直接回答
        messages = [
            {"role": "system", "content": "你是农业专家。"},
            {"role": "user", "content": question}
        ]
        response = llm_client.chat.completions.create(
            model=model_name, messages=messages, temperature=0.5, max_tokens=1024
        )
        return response.choices[0].message.content, []

    # 构建上下文
    context_parts = []
    for doc, meta, dist in knowledge:
        context_parts.append(f"来源: {meta['source']}\n内容: {doc}")

    context = "\n\n---\n\n".join(context_parts)

    system_prompt = """你是农业植保专家。请基于以下参考资料回答用户的问题。
要求:
1. 回答必须基于参考资料
2. 如果资料不足以回答问题, 请如实告知
3. 回答要实用、有针对性"""

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": f"参考资料:\n{context}\n\n问题: {question}"}
    ]

```

```

]

response = llm_client.chat.completions.create(
    model=model_name, messages=messages, temperature=0.3, max_tokens=1024
)

return response.choices[0].message.content, knowledge

# — 主界面 —
if "messages" not in st.session_state:
    st.session_state.messages = []

# 显示历史消息
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

# 用户输入
question = st.chat_input("请输入你的农业问题...")

if question:
    # 显示用户问题
    st.session_state.messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.markdown(question)

    # 生成回答
    with st.chat_message("assistant"):
        with st.spinner("正在检索知识库并生成回答..."):
            answer, knowledge = ask_with_rag(question, top_k)
            st.markdown(answer)

    # 显示参考来源
    if knowledge:
        with st.expander("📖 参考来源"):
            for i, (doc, meta, dist) in enumerate(knowledge):
                st.markdown(f"**来源 {i+1}**": {meta['source']} (相关度: {1-dist:.2f})")
                st.text(doc[:300] + "...")

    st.session_state.messages.append({"role": "assistant", "content": answer})

# — 知识库预览 —
with st.sidebar.expander("📖 知识库文档列表"):
    if collection:
        count = collection.count()
        st.write(f"共 {count} 个文本块")

```

运行应用


```
# 在 Docker 容器中运行（启动时已映射 8501 端口）
streamlit run task_d_challenge.py --server.address 0.0.0.0 --server.port 8501
```

浏览器打开 <http://localhost:8501> 即可使用。

说明：Web 应用在 Docker 容器中运行，使用 agicto 云端 API 作为 LLM 后端。BGE-M3 向量模型会在容器内自动下载（首次可能需要几分钟），之后会缓存到本地。

扩展挑战（选做）

1. **图像识别集成：**在 Web 应用中增加图片上传功能，调用第4部分训练的番茄病害识别模型
2. **多轮对话：**维护对话历史，让 LLM 理解上下文
3. **知识库管理：**添加网页，支持在线编辑、上传新的知识文档
4. **部署上线：**用 ngrok 或云服务器将应用暴露到公网

Git 实践

```
cd /workspace
git init
git add .
git commit -m "Task A: LLM API 调用与 Prompt Engineering"
git add .
git commit -m "Task B: RAG 农业知识库问答系统"
git add .
git commit -m "Task C: vLLM 本地部署与性能测试"
git add .
git commit -m "Task D: 完整的农业 AI 助手 Web 应用"
```

检查清单

完成以下项目后，勾选对应的复选框：

- ☐ 环境准备：成功配置 agicto API，安装所有依赖
- ☐ Task A：完成 API 调用和 3 种 prompt 策略对比实验
- ☐ Task B（数据采集）：用爬虫脚本抓取至少 1 篇农业知识文章
- ☐ Task B（知识库）：构建至少 3 篇农业知识文档的知识库
- ☐ Task B（RAG 系统）：完成检索和问答，输出有/无 RAG 对比
- ☐ Task C：成功用 VSCode Remote-SSH 连接 GPU 服务器
- ☐ Task C：部署 vLLM 服务，完成性能对比测试
- ☐ Task D：运行 Streamlit Web 应用，能正常回答农业问题
- ☐ Git：至少 4 次 commit 记录所有任务

常见问题

Q1: agicto API 调用返回 401 错误

- 检查 API Key 是否正确
- 确认 base_url 为 `https://api.agicto.cn/v1`
- 确认模型名称拼写正确

Q2: BGE-M3 模型下载很慢

- BGE-M3 约 2GB，首次下载需要几分钟
- 模型会缓存到 `~/.cache/huggingface` 目录，后续加载很快
- 如果网络不好，可以配置 HuggingFace 镜像：`export HF_ENDPOINT=https://hf-mirror.com`

Q3: ChromaDB 报错 "collection already exists"

- 这是正常的，说明之前已经创建过
- 代码中已处理这种情况，直接使用现有集合即可

Q4: 向量检索返回的结果不相关

- 检查 BGE-M3 模型是否正确加载
- 尝试调整 chunk_size（建议 300-800）
- 增加 top_k 数量获取更多参考
- 尝试知识库增强中的查询改写策略

Q5: VSCode 无法连接 SSH 服务器

- 确认服务器 IP 和用户名正确
- 确认服务器 SSH 服务正常运行 (`systemctl status sshd`)
- 检查防火墙是否允许 SSH 端口（默认 22）
- 建议先在终端测试 `ssh 用户名@ip` 能否连接

Q6: vLLM 启动后显存不足

- 使用更小的模型（如 7B 而非 14B）
- 添加 `--dtype half` 使用 FP16 精度
- 添加 `--max-model-len 4096` 限制上下文长度
- 关闭其他占用显存的程序

Q7: Streamlit 页面打不开

- 确认 Docker 启动时加了 `-p 8501:8501` 端口映射
- 确认 Streamlit 服务正在运行
- 浏览器访问 `http://localhost:8501`